Projet : Détecteur Anti-Triche et IA pour le Code Source

Analyse complète, Architecture et Guide de Déploiement

Documentation Technique Générée

26 février 2026

Résumé du Projet

Ce document détaille l'architecture, le fonctionnement et la procédure de création du système de détection Anti-Triche. Ce projet utilise du Machine Learning, des techniques d'analyse de code source et une API pour détecter si un code a été écrit par un humain ou généré par une Intelligence Artificielle (ChatGPT, GitHub Copilot).

Table des matières

1 Introduction et Rôle du Projet **3**
1.1 Quel est l'objectif ? . . . 3
1.2 Fonctionnalités Clés . . . 3

2 Architecture et Technologies **3**
2.1 Backend (Serveur et API) . . . 3
2.2 Modèle d'Intelligence Artificielle . . . 3
2.3 Frontend (Interface Visuelle) . . . 4

3 Méthode de Fonctionnement (Workflow) **4**

4 Apprentissage Continu (Le bouton Ré-entraîner) **4**
4.1 Comment le modèle devient "Smart" . . . 4

5 Comment Déployer et Héberger ce Projet **4**
5.1 Étape 1 : Préparation de l'environnement . . . 5
5.2 Étape 2 : Configuration du code . . . 5
5.3 Étape 3 : Secrets (Variables Environnementales) . . . 5
5.4 Lancement avec FastAPI/Uvicorn . . . 5

6 Conclusion **5**

1 Introduction et Rôle du Projet

1.1 Quel est l'objectif ?

Le projet *Anti-Cheat Analyst* a pour vocation de fournir aux écoles, enseignants ou administrateurs de plateformes de recrutement, un outil robuste et automatisé afin d'évaluer l'origine du code source.

Face à la popularité des intelligences artificielles génératives, il est devenu indispensable de distinguer un travail authentique (humain) d'un travail généré par algorithme. L'outil analyse le code et renvoie un verdict de confiance sous la forme d'un score en pourcentage :

- **NE TRICHE PAS** : Code naturel humain.
- **SUSPECT** : Le code présente des motifs algorithmiques (copier/coller massif ou IA partielle).
- **TRICHE PROBABLE** : Le code correspond aux standards d'une IA avec une très haute confiance.

1.2 Fonctionnalités Clés

- **Analyse directe via GitHub** : L'utilisateur n'a besoin que d'entrer l'URL d'un dépôt.
- **Extraction comportementale** : Analyse de la syntaxe (AST), de la complexité cyclomatique, de l'entropie des variables et des commentaires.
- **Détection de plagiat** : Utilisation de modèles TD-IDF pour scanner si le code existe déjà.
- **Apprentissage Continu (Machine Learning)** : Le modèle s'améliore au fil des jours.

2 Architecture et Technologies

Le projet est conçu avec une architecture modulaire pour séparer l'intelligence artificielle, du serveur web et de l'interface graphique.

2.1 Backend (Serveur et API)

Le backend est codé en **Python** avec le framework **FastAPI**. FastAPI a l'avantage d'être extrêmement rapide, asynchrone, et de générer une documentation Swagger native.

- `src/api.py` : Définit les routes de l'API (ex : `/analyze_github`, `/retrain`).
- `src/storage.py` : Gère la persistance des prédictions (JSONL) et les sauvegarde de manière sécurisée sur Hugging Face (Hub).

2.2 Modèle d'Intelligence Artificielle

L'intelligence n'est pas un Large Language Model (comme ChatGPT), mais un modèle de classification **RandomForest** utilisant la librairie **Scikit-Learn**.

- `src/training.py` : Le script d'apprentissage. Il prend des milliers de données étiquetées en entrée pour générer un fichier `model.pkl` (le cerveau mathématique).
- `src/features.py` : C'est le cœur de l'extraction. Au lieu de lire du texte brut, il transforme le code source en un vecteur de nombres décrivant la psychologie du programmeur (Taille des noms de variables, Densité de commentaires, Formatage parfait vs Humain, etc.).

2.3 Frontend (Interface Visuelle)

L'interface utilisateur est construite avec le paradigme du "Glassmorphism" et codée intégralement en **HTML, CSS et JavaScript vanille** sans framework lourd (pas de React ni Vue).

— `src/templates/index.html` : Construit la page moderne, intègre les requêtes fetch() asynchrones pour appeler l'API, et gère les micro-animations.

3 Méthode de Fonctionnement (Workflow)

Voici le chemin exact des données lorsqu'un utilisateur interagit avec le site :

1. **Soumission** : L'utilisateur entre l'URL `https://github.com/user/repo` sur l'interface HTML. Le JavaScript envoie un POST en JSON.
2. **Téléchargement** : Le backend (`api.py`) intercepte l'URL, lit le Token GitHub s'il existe et télécharge les fichiers sources (limités aux 20 premiers) directement via l'API GitHub.
3. **Transformation Vectorielle** : `features.py` agit. Il parse le code avec `ast` (Abstract Syntax Tree) et transforme le texte en chiffres (ex : 20 lignes de code, 3.5 complexité, 12.0 ratio de commentaires).
4. **Prédiction** : Les chiffres sont soumis au `model.pkl` (RandomForest). L'algorithme retourne une probabilité, par exemple : 85% de Triche.
5. **Réponse au Frontend** : L'API retourne les scores par fichier, la moyenne globale, et le verdict final de l'algorithme. Le JavaScript modifie l'affichage de manière asynchrone pour l'utilisateur.

4 Apprentissage Continu (Le bouton Ré-entraîner)

L'une des plus grandes forces du projet réside dans sa capacité à **apprendre au fil du temps** grâce au mode "Fast-Learning" et au système "Human in the Loop".

4.1 Comment le modèle devient "Smart"

— **Auto-Validation** : Chaque fois qu'une URL GitHub est analysée, le modèle enregistre instantanément sa propre prédiction. À présent (après des mises à jour récentes), le modèle collecte automatiquement ces prédictions pour pouvoir s'adapter à de nouveaux langages et à de nouveaux patterns.

— **Bouton d'Administration** : L'interface intègre un bouton "Ré-entraîner" caché par mot de passe (par défaut `NEGzaoui1234`).

— Lorsqu'il est déclenché, le serveur lance la commande de ré-entraînement : il ajoute les nouveaux exemples collectés à sa base de données existante. L'algorithme se reconstruit, compare son ancienne précision à la nouvelle, et se met à jour instantanément sans redémarrage.

5 Comment Déployer et Héberger ce Projet

L'outil complet a été pensé pour être conteneurisé et très facilement déployable sur la plateforme d'intelligence artificielle **Hugging Face Spaces**.

5.1 Étape 1 : Préparation de l'environnement

— Avoir un compte sur Hugging Face.
— Créer un nouveau "Space" et lui attribuer un kit de déploiement (Docker ou Gradio/FastAPI natif).

5.2 Étape 2 : Configuration du code

Le projet doit idéalement contenir le fichier principal (`app.py` ou `src/api.py`) et un fichier des dépendances (`requirements.txt`) contenant :

```
fastapi
uvicorn
scikit-learn
pydantic
requests
huggingface_hub
```

5.3 Étape 3 : Secrets (Variables Environnementales)

Pour que le conteneur fonctionne pleinement et surpasse les limites, vous devez ajouter en secret dans votre espace Hugging Face :
— `GITHUB_TOKEN` : Un token d'accès personnel GitHub (PAT) classique pour éviter l'erreur de "Rate Limit" (bloqué à 60 requêtes/heure) et lire les dépôts privés.
— `HF_TOKEN` / `HF_REPO` : Utilisés pour que le conteneur Push ses propres nouvelles versions de modèles après un ré-entraînement sur le cloud.

5.4 Lancement avec FastAPI/Uvicorn

Si le code est hébergé en local pour des tests, un terminal suffit :

```
uvicorn src.api:app --host 0.0.0.0 --port 7860
```

Le port 7860 est le port standard recherché par les interfaces Hugging Face ou Gradio. La page sera accessible sur `http://localhost:7860`.

6 Conclusion

Le projet Anti-Cheat est un dispositif puissant et autonome. Entièrement développé en Python, combinant le natural language processing par expressions régulières, des arbres de décision statistiques (Random Forest), et un serveur web multi-threads (Uvicorn/FastAPI). Sa grande autonomie, grâce à ses algorithmes de "ré-apprentissage", en fait un produit logiciel évolutif et prêt pour une utilisation réelle en production de type "Salle de classe" ou "Plateforme de Challenges".